Auction e-Commerce System

Team 9: Code2Cash

EECS 4413: Deliverable 2

Winter 2026

Project Github Repository:

https://github.com/PeytonHislop/EECS4413Auction_e-Commerce_System

Ravneet Deol - 217270067

Syed Mustafa Jamal - 217761651

Anton Caushaj - 219468024

Peyton Hislop - 217342957

Document Change Control Revision History

Version	Date	Author(s)	Summary of Changes
1.0	Jan 25, 2026	Ravneet Deol	Created the document outline
1.1	Feb 2, 2026	Ravneet Deol	Update to use cases, architecture - services, introduction - references, major design decisions,
1.2	Feb 2, 2026	Syed Mustafa Jamal	Edited document formatting, added UC's related to auction service, added base/structure for auction diagrams, added comments for group clarifications
1.3	Feb 3, 2026	Ravneet Deol	Update to use cases, major design decisions - architectural patterns chosen, test-driven development
1.4	Feb 4, 2026	Anton Caushaj	Completion of the Project Purpose/Overview description, updates to use cases, and the addition of final drafts of the Catalogue Activity and Sequence diagrams. Major design decisions added: Catalogue Test Driven Development, Catalogue Architecture. Added requested items to the product backlog.

1.5	Feb 4, 2026	Ravneet Deol	Update to test-driven development, use cases, major design decisions, product backlog,
1.6	Feb 5, 2026	Ravneet Deol	Sequence diagram, Activity diagram, Architecture - Interfaces, Major design decisions - architectural patterns chosen
1.7	Feb 5, 2026	Peyton Hislop	Sequence, activity diagram, payment service use cases
1.8	Feb 6, 2026	Peyton Hislop	Added Services, interfaces and test cases
1.9	Feb 6, 2026	Syed Mustafa Jamal	Added sequence, activity & architecture & UC test-cases, added architecture C, added acknowledgements, formatted document and adjusted sizes of charts for better clarity
2.0	Feb 6, 2026	Ravneet Deol	Updated architectural patterns chosen, architectural patterns not chosen, completed Gantt diagram, completed component diagram
2.1	March 06, 2026	Ravneet Deol	Added an implementation updates section. Updated test cases, services, and interfaces. Added to the AI disclosure section.

2.2	March 6, 2026	Anton Caushaj	Test case updates, implementation updates for the Catalogue service and project design patterns chosen/not chosen.
2.3	March 6, 2026	Syed Mustafa Jamal	Added implementation updates for Auction service, test case updates.
2.4	March 6, 2026	Peyton Hislop	Added implementation updates for Payment service.

Table of Contents

Document Change Control Revision History.....2
Table of Contents.....5
Introduction.....6
Major Design Decisions.....7
Modularization Criteria:.....7
Architectural patterns chosen:.....7
Architectural pattern not chosen:.....8
Implementation Updates.....9
IAM implementation summary.....9
Catalogue implementation summary.....9
Auction implementation summary.....10
Payment implementation summary.....11
Use Cases.....12
Identity and Authorization Management (IAM) Service Use Cases:.....12
Auction Service (AS) UCs:.....12
Catalogue Service UCs:.....13
Payment Service UCs:.....13
Sequence Diagrams.....14
Activity Diagrams.....17
Component Diagram.....20
Architecture.....21
Services:.....21
Interfaces:.....22
Activities Plan, Product Backlog, and Sprint Backlog.....29
Product backlog:.....29
GANTT diagram:.....31
Test Driven Development.....32
Acknowledgements.....49

Introduction

Purpose: The general objective of this project is to design and implement a robust e-commerce auction system using open-source web-based frameworks. More specifically, the project focuses on a Forward Auction where an item is placed for sale by a seller and a buyer must enter increasingly strict bids until the time runs out (at which point the item will no longer be available). Furthermore, this project integrates architectural goals beyond simple programming constructs, such as intended microservice deployments and a modular design through the separation of business logic, request operations, and front-end implementations.

Overview: This software application helps fix the problem of having a secure online auction in real time at a distance from seller to buyer. It essentially creates a marketplace where all users can access the same list of items for available bidding, track live bidding wars, and securely process payments for won items. The overall system requirements mandate a three-tier architecture:

Front-end: An HTML/CSS-based interface for user interaction.

Middle-tier: Framework-based layer (e.g. SpringBoot), which allows for swift relay requests and API communication.

Back-end: Independent services containing business logic and Database integration/connectivity.

This is a microservices architecture. Each service runs in its own process, meaning that IAM, Catalogue, Auction and Payment have separate databases and they communicate with each other through open standards (e.g., HTTP/REST) to fulfill application requirements.

Overall, the system must support core functionalities, including User Identity Management (Sign-Up/Sign-In), Catalogue Management (Search and Display), Auctioning Engines (validating bids), and Payment Processing.

References:

- EECS 4413 Project Description: Building e-Commerce Systems. York University, 2026.
- Forward Auction. Wikipedia. (Referenced in Project Description)
- *Gantt Chart*. Wikipedia. (Referenced in Deliverable 1 Instructions)

Major Design Decisions

Modularization Criteria:

- High cohesion
    - Our system's design focuses on keeping each module focused on one primary function. This allows each service to have a clear purpose. For example, the Identity and Authorization Management Service focuses on only identity and authentication, password reset, and retrieval of a user's profile. This improves quality because each service has a narrow focus, is able to be tested independently, and can be changed without affecting unrelated functionality.
- Low coupling
    - Our system's design minimizes the dependencies that exist between modules by limiting how each service interacts with one another. Services will communicate with each other via a clear set of REST endpoints that other services can call. The gateway service will act as a single entry point for clients to keep the internal service structure isolated from the front-end. Additionally, each service will own its own database and will not be able to write directly to another service's database.

Architectural patterns chosen:

1. We will adopt a three-tier architecture to separate concerns.
    - Front-end subsystem - HTML/CSS/JS pages
    - Middle subsystem - Gateway service will receive UI requests and orchestrate calls to backend services
    - Back-end subsystem - Contains business logic and data persistence
2. Microservices-oriented modular design
    - Each service will run independently and have its own datastore. Supports parallel team development. Each service will be able to be tested independently and evolve separately.
3. API Gateway/Facade
    - Gateway functions as the UI's stable entry point. Will enforce token checks. The UI will stay simple, and the system will avoid exposing internal details to the front-end.
4. DAO Pattern

    - Each service will use DAOs to isolate how they interact with persistence logic. Will keep SQL details away from business logic. Will allow persistence to be changed more easily in the future if needed without having to rewrite domain logic.
5. Pub-Sub
    - The methods through which users will receive real-time bid updates. Will allow all users to see the current highest bid without requiring constant page refreshes.
6. Dependency Injection
    - Spring's @Autowired keyword throughout allows the promotion of loose coupling, testability (referring to @MockBean), and the overall lifecycle management of the different components.

Architectural pattern not chosen:

- Decided against the usage of a message broker such as Kafka at the moment. The pros of using such a pattern would improve decoupling and scaling for our project, but it would add complexity. Instead, the project will focus on using REST and WebSocket for real-time updates. In the future, if we believe the benefits outweigh the cons for deliverables 2 and 3, we may reconsider our thoughts on this choice.
- We also decided against the usage of the Circuit Breaker pattern. Omitting the use of the Breaker pattern is an intentional architectural decision that prioritizes simplicity and learning. Since the services are running on a local host with no actual network failures, and the services have a hard dependency on each other, implementing Circuit Breakers would only add unnecessary complexity without solving the problems that arise in the completion of the project. The fail-fast behaviour enables immediate debugging of issues rather than masking them with complex logic, making it easier to understand how the microservices interact/depend on each other.

Implementation Updates

IAM implementation summary

- Tech stack: Spring Boot 4.0.3, Java 21, Spring Web/Security/JPA/Validation, H2, JJWT
- What's implemented: UC-IAM-01 to UC-IAM-06
- How other services use it: pass JWT in Authorization: Bearer <token>; use /auth/validate and /auth/authorize; use /users/{userId} for shipping profile.

Catalogue implementation summary

- Tech stack: Java 21 (LTS), Maven 3.9.12 (build tool), Spring Boot 3.2.0, JUnit 5, Spring Boot Test, JaCoCo, Port: 8083
- Implemented: UC-CAT-2, UC-CAT-2.1, UC-CAT-7
- How other services use it: The Gateway service acts as a proxy, where client requests are forwarded from /api/items/* using the RestClient, using the port 8083. The Catalogue service validates authentication by calling the IAM service at 8081 using RestTemplate to check the JWT tokens and verify the seller roles before actually allowing the item creation.
- Design Patterns implemented: **Microservice architecture** - Services communicate via synchronous HTTP/REST calls. **Facade Pattern** - CatalogueFacade provides an interface to complex subsystems, such as logic filtering, repository interactions, encapsulations of data such as date calculations, and repository interactions from the controller layer. **Dependency Injection** - Spring's @Autowired keyword throughout allows the promotion of loose coupling and testability (referring to @MockBean).

Auction implementation summary

- Tech stack: Spring Boot 3.2.0, Java 17, Spring Web/JDBC/Validation, SQLite 3.44.1, WebFlux (for inter-service HTTP calls), JdbcTemplate
- Database: : SQLite with schema auto-initialization on startup, separate database file
- Port: 8082
- What's implemented: UC1-AS through UC4-AS - auction and bid management lifecycle
- How other services use it:
    - Gateway Service routes all /api/auctions/* requests to this service (port 8080 → 8082), Public endpoints (GET active auctions, bid history) require no authentication, Protected endpoints (POST auction, POST bid) require JWT in Authorization: Bearer <token> header, Service validates tokens and roles via IAM service endpoints /auth/validate and /auth/authorize, Automatic auction closure via scheduler (runs every 60 seconds, checks for expired auctions), Calls Catalogue service to verify item exists before creating auction, Calls Payment service when auction closes with a winner
- Design patterns implemented:
    - DAO Pattern - Separates database access from business logic
    - DTO Pattern - Clean API contracts with Jakarta validation
    - Service Layer Pattern - Business logic isolated from controllers
    - MVC Pattern - Clear separation of Controller → Service → DAO
- Key features:
    - RESTful API with 11 endpoints, JWT authentication and role-based authorization
    - Real-time bid validation (must exceed current highest), Automatic auction closure and winner determination, Comprehensive error handling and validation, Database persistence with SQLite

Payment implementation summary

- Tech stack: Java 17, Maven 3.9.12 (build tool), Spring Boot 3.1.5, Port: 8084
- SQLite with automatic entity mapping, persistence of payments and orders and schema generation
- Gateway Integration: Payment Service does not directly communicate with other services the gateway gets required data before sending the request
- Port 8084
- One api endpoint POST /api/payments/process that handles receipt generation based on auction id. Where card info is entered
- IAM Integration the system uses JWT-based authentication Payment requests must contain Authorization: Bearer <JWT>
- This service uses all other services through the gateway to get data and generate receipt
- Design patterns implemented: MVC Pattern - Clear separation of Controller → Service → Repository → Database. Repository Pattern uses repository classes/interfaces to isolate database access (PaymentRepository, OrderRepository). DTO Pattern used to separate objects for request/response transfer instead of exposing entities directly. Facade Pattern (PaymentClient) acts like facade over more complicated subsystem. API Gateway Pattern validating the JWT with IAM and obtaining data from other services through the gateway this gives cleaner inter-service communication.
- What's implemented: UC-PAY-1 and UC-PAY-2 - payment handling and receipt generation

Use Cases

Identity and Authorization Management (IAM) Service Use Cases:

UC-IAM-01: Sign-Up

- Description: A user creates a new account

UC-IAM-02: Sign-In

- Description: A user's username and password are verified to start a session

UC-IAM-03: Forgot Password/Reset Password

- Description: A user is able to reset their login credentials if forgotten

UC-IAM-04: Validate Session/Token

- Description: Every protected action will confirm that the user is logged in.

UC-IAM-05: Authorize Role-Based Actions

- Description: Decides if a user is able to perform an action based on the scope of actions they're limited to in their role.

UC-IAM-06: Provide User Profile For Payment

- Description: The payment page must show the correct user's information (shipping address).

Auction Service (AS) UCs:

UC1-AS: Create Auction Listing

Description: A seller creates a new auction listing for an item they want to sell

UC2-AS: Place Bid on Auction

Description: A buyer places a bid on an active auction

UC3-AS: Close Auction and Determine Winner

Description: The system automatically closes an auction when the time expires and determines the winner

UC4-AS: View Active Auctions

Description: A user browses all currently active auctions

Catalogue Service UCs:

UC-CAT-2: Browse the Catalogue of Auctioned Items

UC-CAT-2.1: Item Search

Description: Once logged in, the user provides a keyword, and a list of items up for auction (or currently being auctioned) matching the keywords are displayed.

Responsibility: The service queries the database for item descriptions matching the keyword and filters out items that are no longer active.

UC-CAT-7: Auction Item (Static Catalogue)

Description: A seller uploads information to the system to list an item for sale.

This use case handles the Content Creation (what is being sold), while UC1-AS handles the Market Initialization (opening the bidding floor).

Responsibility: The Catalogue Service validates the seller's input (Description, Forward Auction Type, Duration, Shipping Price) and calculates the absolute End Date by adding the duration to the current timestamp. It persists this static record to the Catalogue Database.

Payment Service UCs:

UC-PAY-1: Payment Page

Description: Once the winner selects the “Pay Now” button, the winning bidder is served with a page to pay with the user's address from the database, the item price and shipping cost. The user adds the credit card number, the name on the card, the expiration date and the security code.

UC-PAY-2: Receipt Page and Shipping details

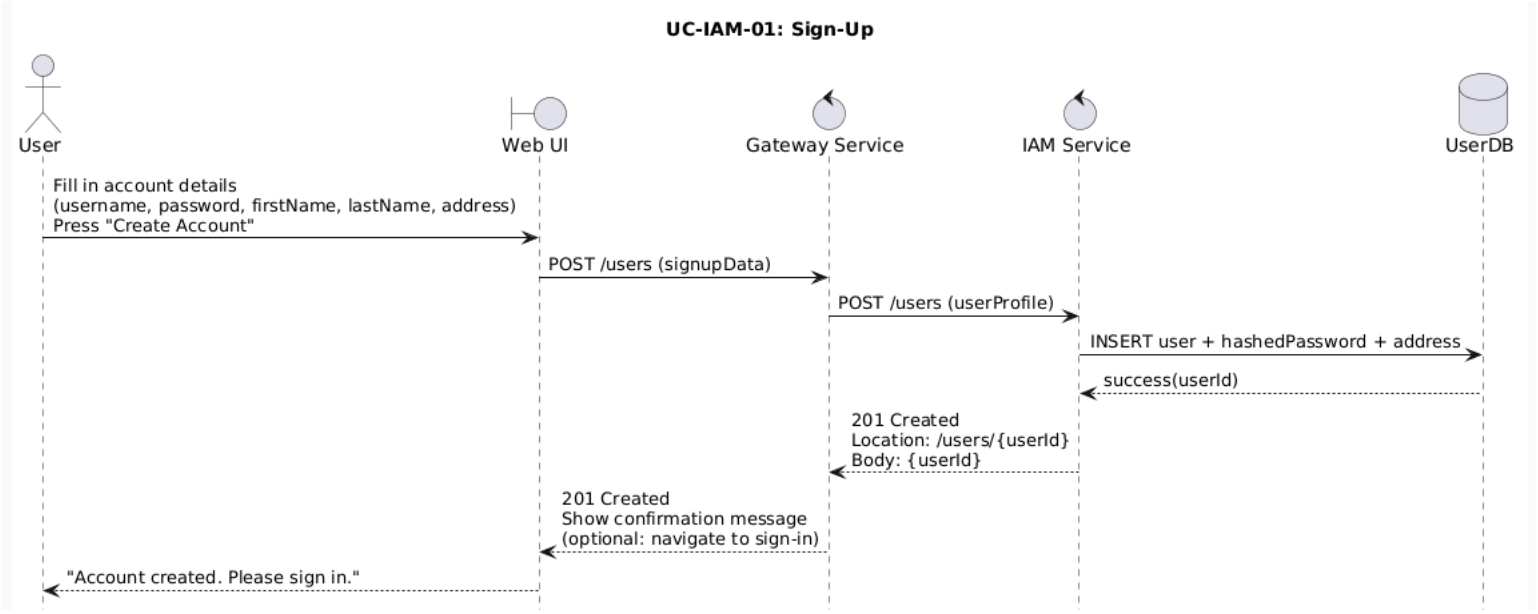
Description: Once payment is cleared, the winning bidder is served with a receipt and shipment information. The page provides the total amount paid as well as the number of days shipping will take.

Sequence Diagrams

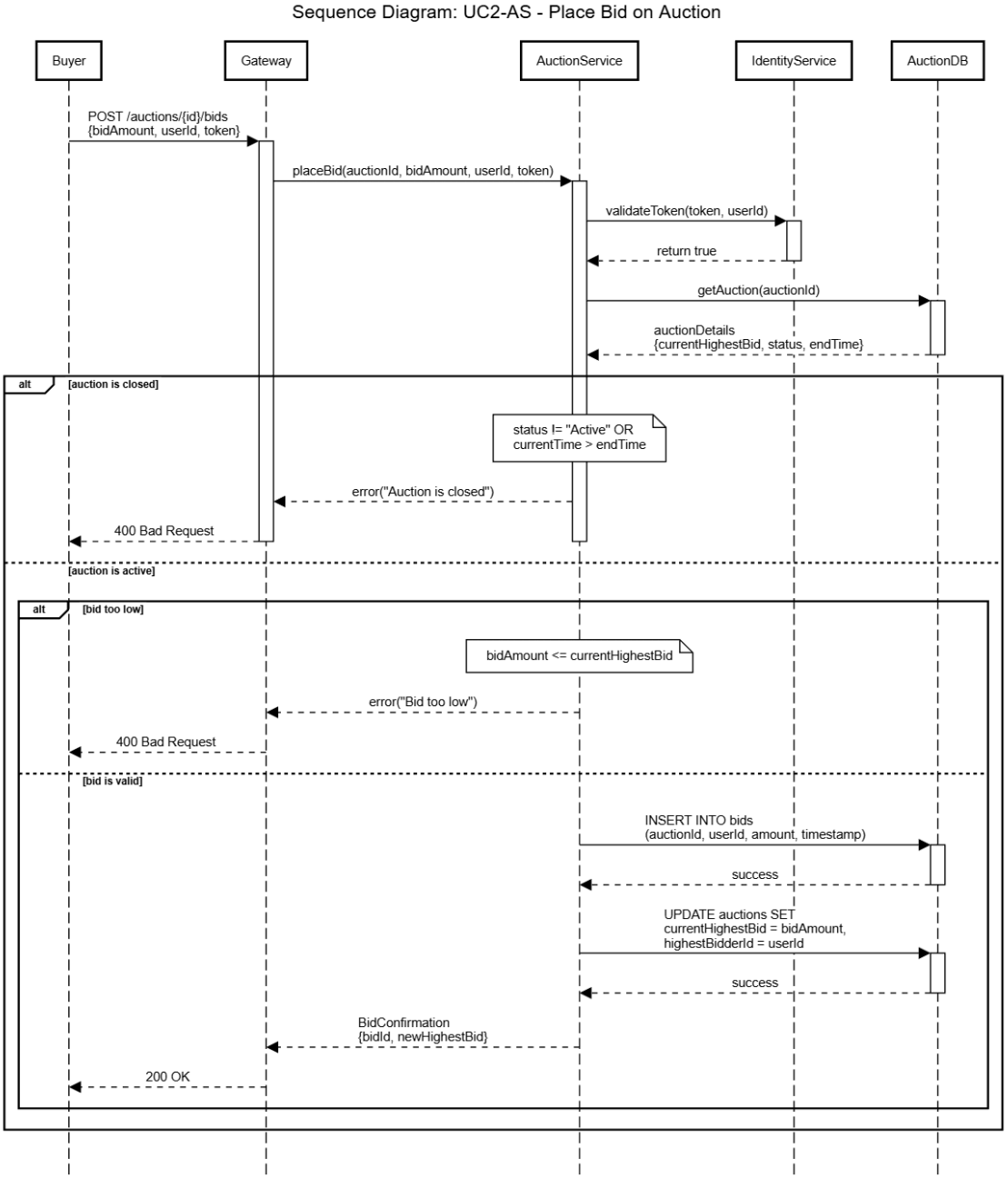
Identity and Authorization Management Service Sequence Diagram for UC-IAM-01:

<u>Legend</u>

Stick figure (Actor)	External user that interacts with the system
Participant header (Web UI, Gateway Service, IAM Services, UserDB)	A system component that sends and receives messages
Vertical dashed line	The time span that a participant exists for during interactions.
Horizontal solid arrow	A request being sent
Horizontal dashed arrow	A response being returned
Message label text	Description for actions being performed



Auction Service Sequence Diagram, Use Case: UC6 - Place Bid on Auction



Legend: (for the Auction diagram above, Catalogue diagram and Payment diagrams below)

Solid Arrow (→): Synchronous method call or request

Dashed Arrow (- - →): Return value or response

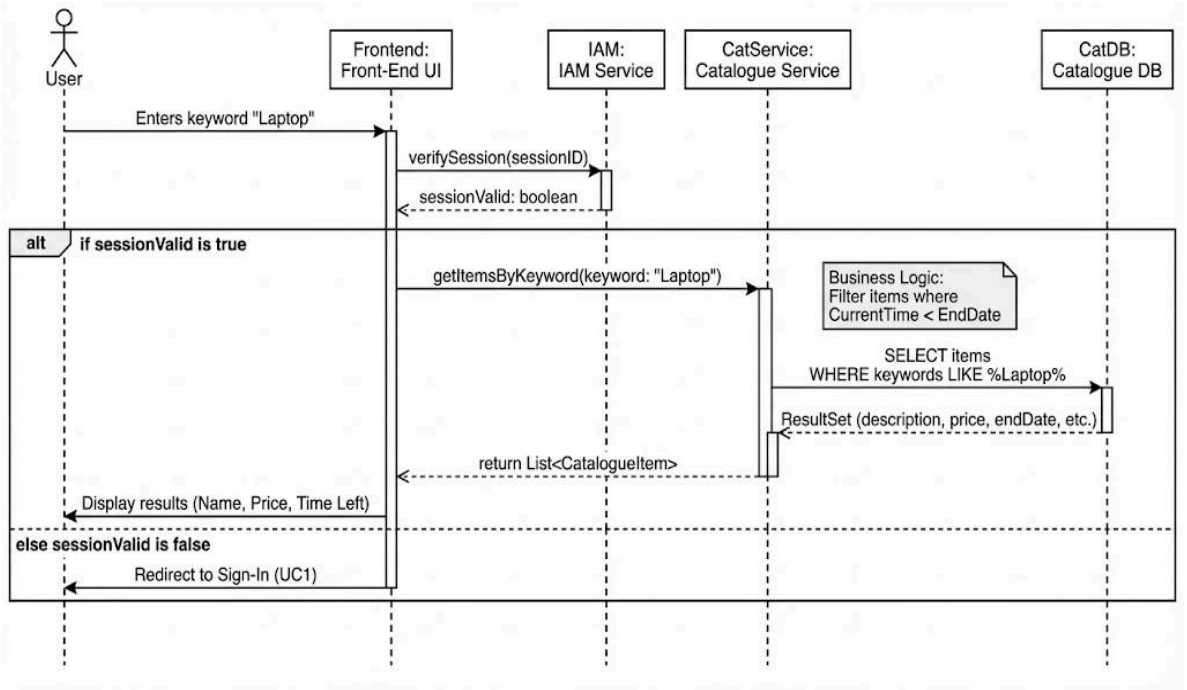
Vertical Rectangle: Activation period (component is processing)

Vertical Dashed Line: Lifeline (component exists)

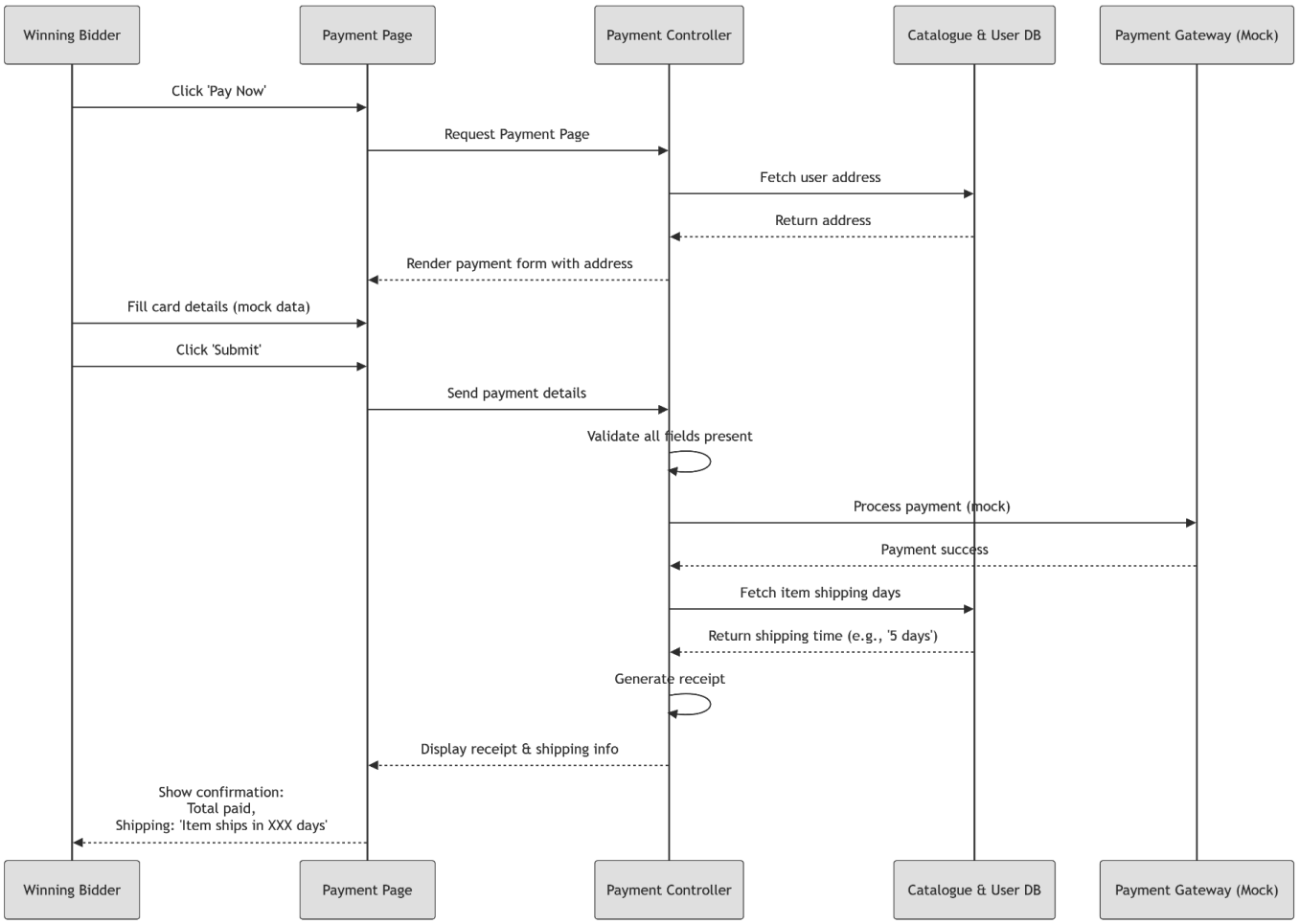
Alt Box: Conditional logic with alternative paths

Note Box: Explanation of conditions or business logic

Sequence Diagram for Use Case UC-CAT-2.1 - Item Search:



Sequence Diagram for Use Case UC-PAY-1, UC-PAY-2



Activity Diagrams

Identity and Authorization Management Service Components Diagram for UC-IAM-01:

<u>Legend (Applies for IAM service and catalogue service activity diagrams)</u>

Rounded rectangle - Action/Activity step.

Swimlane - What service is performing the steps.

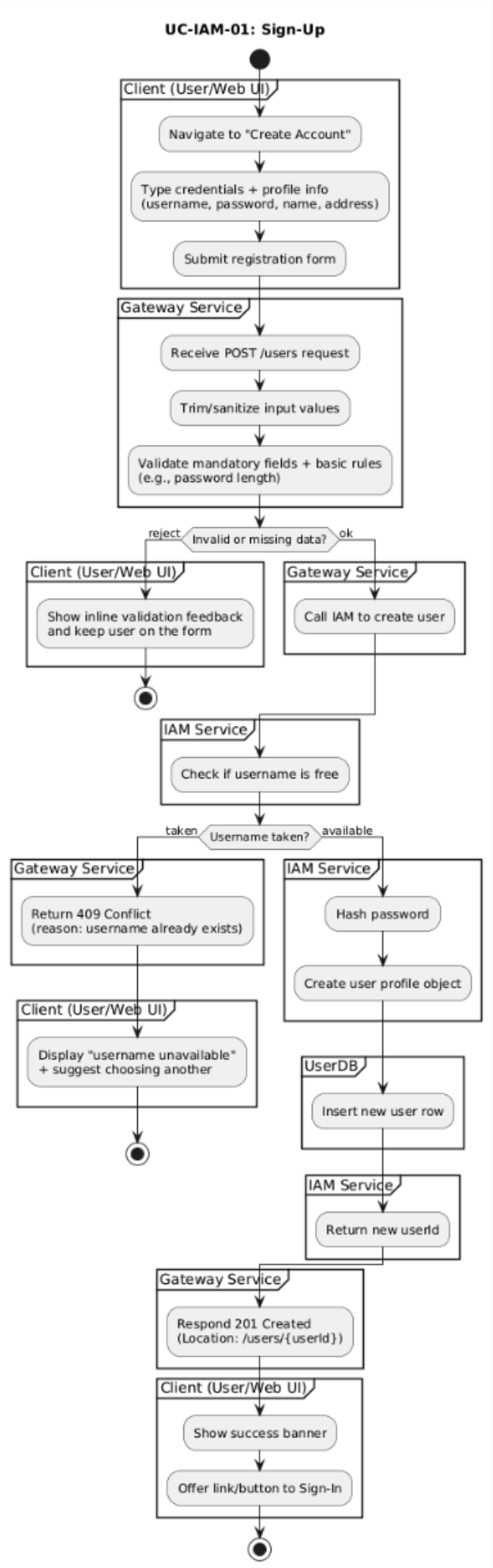
Solid arrowed line - Control flow.

Diamond: A branch point that picks a path based on a condition.

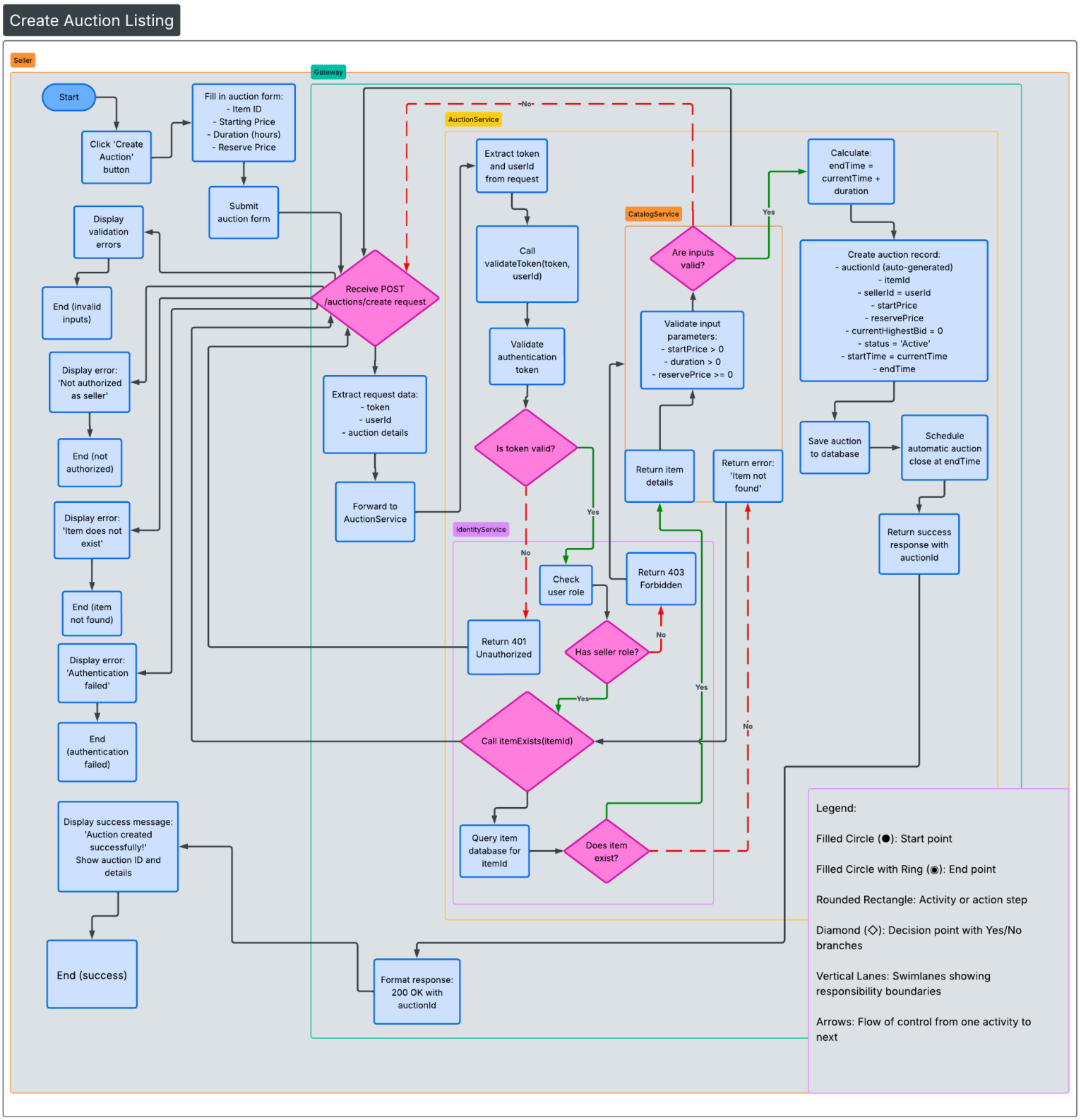
Decision labels: Condition outcomes that select outgoing branches.

Black circle - Where the process starts.

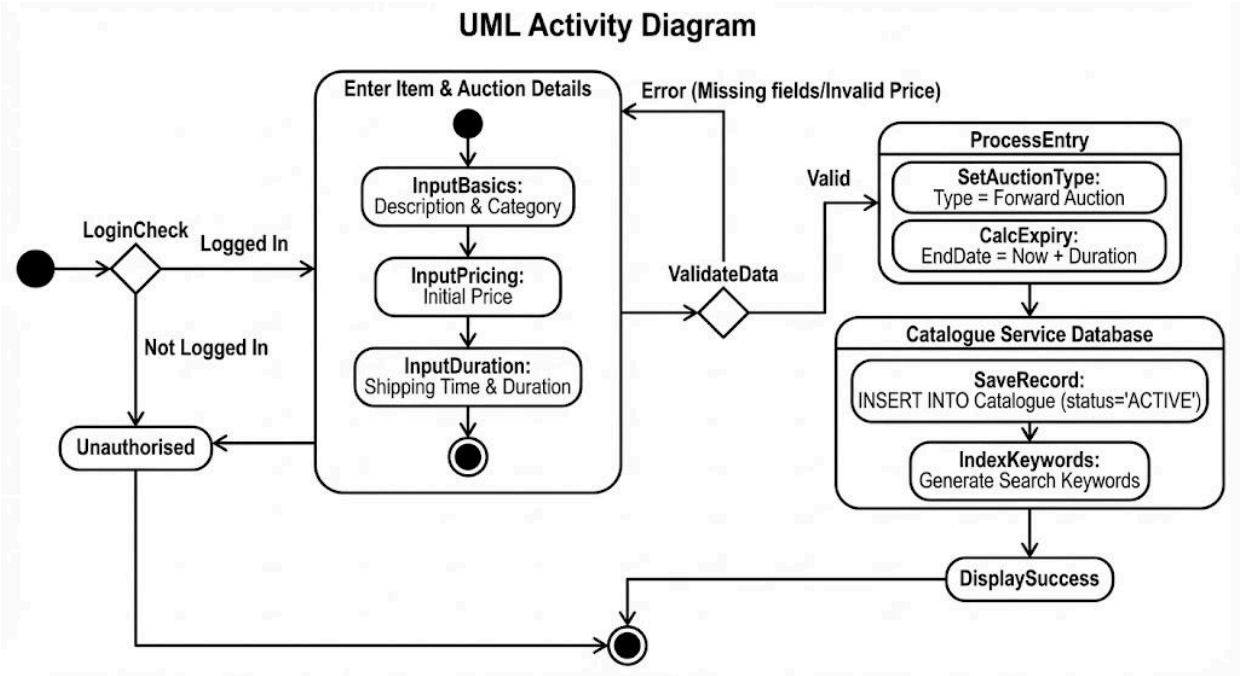
Target circle - Where the process terminates.



Auction Service Activity Diagram, Use Case: UC5 - Create Auction Listing



Activity Diagram for UC-CAT-7 - Auction Item (Static Catalogue) Listing:



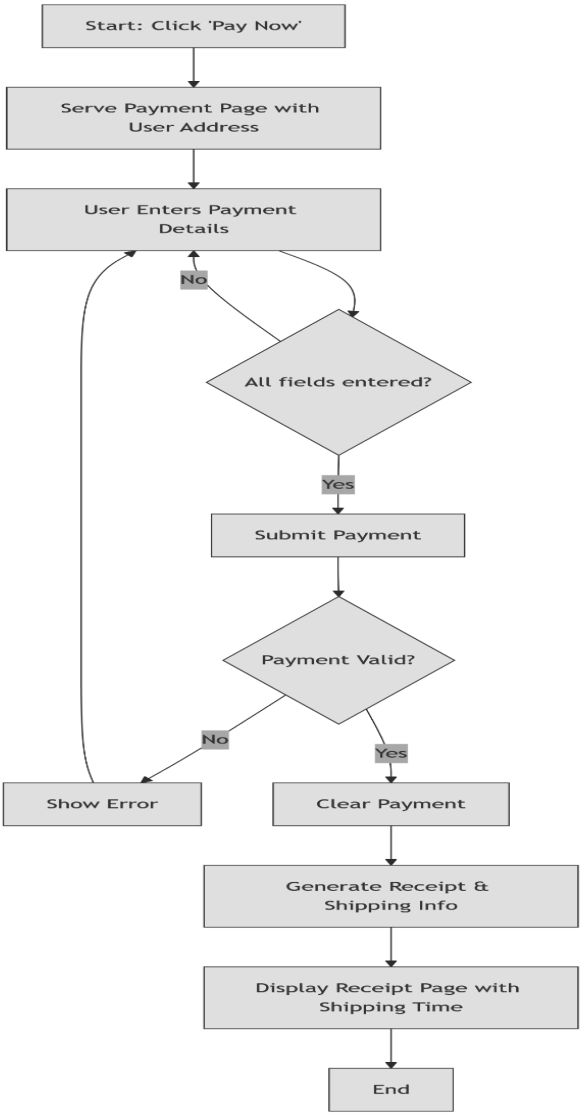
Activity Diagram for UC-PAY-1, UC-PAY-2

Legend:

Rectangle: Action/Activity

Diamond: Decision Node

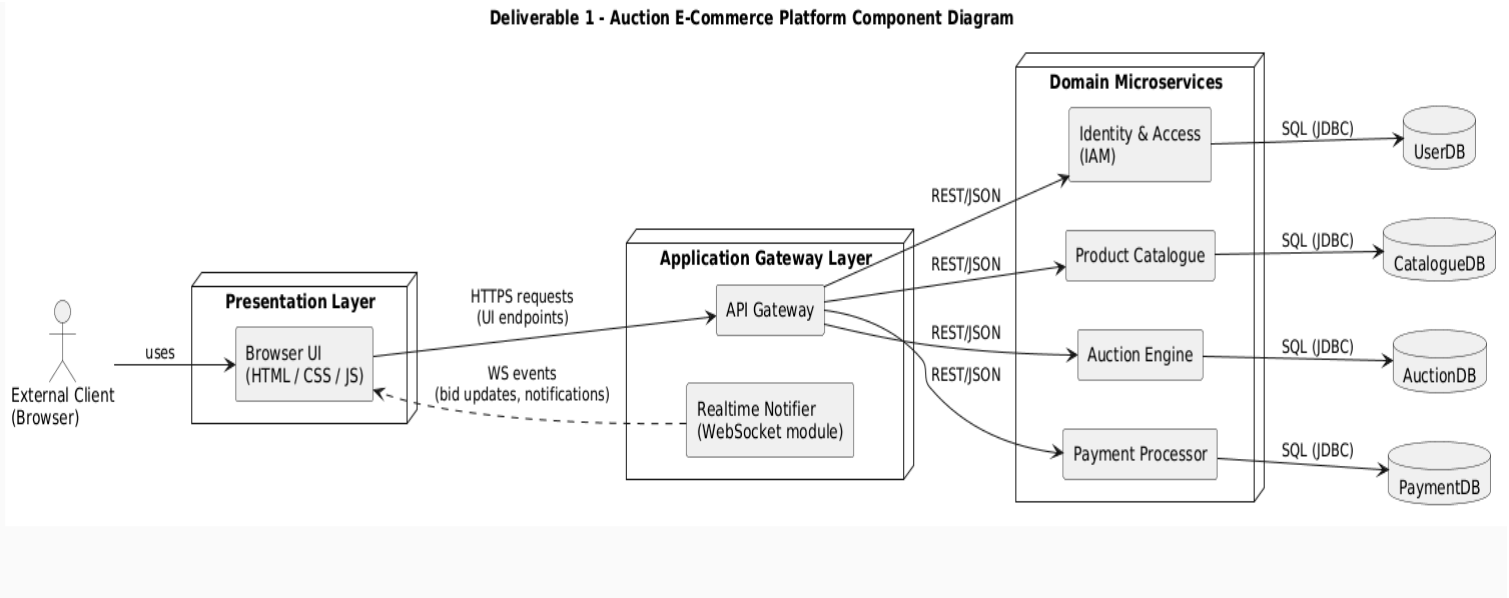
Arrows: Flow/Transition



Component Diagram

<u>Legend</u>

Stick figure (Client)	End user that is outside of the system boundary
Large labeled boxes	Subsystem boundaries
Smaller boxes instead of layers	Components/services deployed within layers.
Database cylinder icons	Data stores used by the services
Solid arrows labeled “REST/JSON”	Synchronous API calls
Lines labeled “HTTPS requests”	Browser-to-gateway communications for routes/pages that interact with the UI
Dashed arrows labeled “WS events”	Asynchronous push-style messages
SQL (JDBC) labels	Service-to-database connectivity



Architecture

Services:

Service Name	Description	Exposed Interface Names	Interface Description
Gateway Service	Façade / entry-point that forwards /api/* requests to downstream services.	GW:AuthGatewayController GW:UsersGatewayController GW:HealthController	GW:AuthGatewayController – /api/auth/* proxy routes to IAM /auth/*. GW:UsersGatewayController – /api/users/* proxy routes to IAM /users/*. GW:HealthController – /api/health for gateway status.
Identity and Authorization Management Service	Manages user sign-up, sign-in, token validation, role authorization, password reset, and user profile retrieval (shipping address).	IAM:AuthController IAM:UserController	IAM:AuthController – /auth/* endpoints for authentication, token validation, role authorization, and password reset. IAM:UserController – /users/* endpoint for user profile retrieval.
Catalogue Service	Manages the lifecycle of auction items, including validating new	CatalogueFacade	Provides a simplified interface for the Front-End and Gateway

	'Forward Auction' listings, storing item descriptions/shipping costs, and enabling keyword-based search functionality.		to search for items, retrieve item details, and validate new seller listings. This interface will also be very important in terms of validating different functions without exposing the database logic.
Payment Service	Handles payment processing, receipt generation and shipping information for the user after the auction is won.	PaymentManagement OrderManagement ShippingManagement ReceiptManagement	PaymentManagement- Process payments securely OrderManagement- Manages order lifecycle ShippingManagement- Handles shipping information and delivery time ReceiptManagement- Delivers receipt page with details
Auction Service	Manages the auction lifecycle, such as: creating listings, processing bids, tacking highest bids, closing auctions and determining winners.	AuctionManagement BidManagement	AuctionManagement - Endpoints for creating, retrieving and closing auctions BidManagement - Endpoints for placing and validating bids.

Interfaces:

Interface Name	Operations	Operation Descriptions
GW:HealthController	Map health() → GET /api/health	Returns {status: "gateway ok"} to confirm gateway is running.

IAM:AuthController	1) signup Map<String,Object> signup(SignupRequest req) → POST /auth/signup (201 Created) 2) login AuthResponse login(LoginRequest req) → POST /auth/login (200 OK) 3) validate ValidateTokenResponse validate(String Authorization) → POST /auth/validate (200 OK) 4) forgotPassword Map<String,String> forgotPassword(ForgotPasswordRequest req) → POST /auth/password/forgot (200 OK) 5) resetPassword Map<String,String> resetPassword(ResetPasswordRequest req) → POST /auth/password/reset (200 OK)	1) POST /auth/signup (UC-IAM-01) Input: JSON SignupRequest (validated with @Valid): username, password, firstName, lastName, email, role, shippingAddress{streetNumber, streetName, city, country, postalCode}. Processing: creates user, hashes password, enforces unique username/email. Success (201): JSON { userId, username, role, message }. Failure: 400 validation errors (missing/blank fields), 409 conflict on duplicate username/email (as implemented in your service/exception handling). 2) POST /auth/login (UC-IAM-02) Input: JSON LoginRequest (validated): username, password. Processing: verifies password hash; issues JWT. Success (200): AuthResponse JSON { token, tokenType, expiresIn, userId, username, role }. Failure: 401 if credentials invalid; 400 if request invalid (missing fields). 3) POST /auth/validate (UC-IAM-04) Input: Header Authorization: Bearer <jwt> (required).

	6) authorizeRole Map<String,Object> authorizeRole(String Authorization, Role requiredRole) → GET /auth/authorize?requiredRole=... (200 OK)	Processing: extractBearer() checks header format; validates JWT signature/expiry; extracts claims. Success (200): ValidateTokenResponse JSON { valid, userId, username, role } (valid=true for good token; valid=false for invalid/expired token depending on your service logic). Failure: If header missing/doesn't start with Bearer → throws IllegalArgumentException (commonly mapped to 400 unless you map differently). 4) POST /auth/password/forgot (UC-IAM-03) Input: JSON ForgotPasswordRequest (validated): typically username (per your script). Processing: generates reset token linked to user and stores it (one-time / expirable depending on model). Success (200): JSON { message: "Password reset token generated", resetToken }. Failure: 400 if request invalid; may be 404 if username not found (depends on your service behavior). 5) POST /auth/password/reset (UC-IAM-03) Input: JSON ResetPasswordRequest (validated): resetToken, newPassword. Processing: verifies reset token (exists, not used, not

		expired), updates password hash, marks token used. Success (200): JSON { message: "Password reset successful" }. Failure: 400 invalid/missing fields; 401/404/409 depending on how you signal invalid/expired/used reset tokens (document what your service currently returns). 6) GET /auth/authorize (UC-IAM-05) Input: Header Authorization: Bearer <jwt> and query param requiredRole mapped to enum Role (required). Processing: validates JWT; compares user role claim vs requiredRole. Success (200): JSON { authorized: <true/false>, requiredRole: "<ROLE>" }. Failure: 400 if requiredRole missing/invalid enum value; 400 if Authorization header missing/invalid prefix (via extractBearer()).
IAM:UserController	UserProfileResponse getUserProfile(String userId) → GET /users/{userId}	getUserProfile – Returns profile including shipping address (used by Payment).
CatalogueFacade	List<Item> CatalogueFacade:getItemsByKey	Goes through the database for items matching the keyword and filters out expired auctions.

	word(String keyword)	
	int CatalogueFacade:addItem(Item DTO details)	Validates input (positive price, valid duration), calculates the End Date, and adds the new item to the DB.
	Item CatalogueFacade:getItemDetails (int itemID)	Retrieves the full details (static) (Description, Shipping Cost, Initial Price) for a single item page.
AuctionManagement	AuctionResponse createAuction(AuctionDetails details, String token) used by the Gateway.	AuctionManagement: createAuction() - Validates seller credentials via Identity Service, validates item exists via Catalogue Service, creates new auction record with calculated end time, and schedules automatic closure.
	Auction getAuction(String auctionId) used by Gateway, BidManagement.	AuctionManagement: getAuction() - Retrieves details of a specific auction, including current highest bid, status, and time remaining
	AuctionResponse closeAuction(String auctionId) used by System Scheduler.	AuctionManagement: closeAuction() - Changes auction status to "Closed", determines winner if reserve price met, notifies winner and seller, initiates payment process
	List<Auction> getActiveAuctions() used by Gateway	AuctionManagement:getActiveAuctions() - Returns list of all auctions with status "Active" and end time not yet reached

BidManagement	BidResponse placeBid(String auctionId, double bidAmount, String userId, String token) used by Gateway	BidManagement: placeBid() - Validates user token via Identity Service, checks auction is active, validates bid amount is higher than current highest bid, records bid with timestamp, updates auction's current highest bid
	List<Bid> getBidHistory(String auctionId) used by Gateway	BidManagement: getBidHistory() - Returns a chronological list of all bids placed on a specific auction
	Bid getHighestBid(String auctionId) used by AuctionManagement	BidManagement: getHighestBid() - Returns the current highest bid for an auction, including bidder ID and bid amount
PaymentManagement	initiatePayment(userId, itemId, amount) validatePaymentDetails(paymentRequest) processPayment(paymentRequest) verifyPaymentStatus(transactionId)	initiatePayment() Starts the payment validatePaymentDetails() Checks if payment details are valid processPayment() Processes payment verifyPaymentStatus() Verifies payment status
OrderManagement	createOrder(userId, itemId, paymentDetails) updateOrderStatus(orderId, status) getOrderDetails(orderId)	createOrder() Creates order updateOrderStatus() Updates the status of the order getOrderDetails() Gets details of the order
ShippingManagement	getShippingTime(itemId) calculateShippingCost(itemId, address)	getShippingTime(itemId) Gets shipping time calculateShippingCost() Calculates shipping cost

ReceiptManagement	generateReceipt(orderId, paymentDetails) sendConfirmation(userId,receiptData)	generateReceipt() Generates receipt page with shipping details sendConfirmation() Sends confirmation of shipping to user

Activities Plan, Product Backlog, and Sprint Backlog

Product backlog:

EPIC A - Identity and Authorization Management

ID	Task
A1	Sign-Up (UC-IAM-01)
A2	Sign-In (UC-IAM-02)
A3	Reset Password (UC-IAM-03)
A4	Validate Session/Token (UC-IAM-04)
A5	Authorize role-based actions (UC-IAM-05)
A6	Make user profile retrievable (UC-IAM-06)

EPIC B - Catalogue Service

ID	Task
B1	DB Schema Design (UC-CAT-7)
B2	Item Creation API (UC-CAT-7)
B3	Keyword Search API (UC-CAT-2.1)
B4	Item Detail API (UC-CAT-2)
B5	Expiration Filtering Logic (UC-CAT-2.1)
B6	Containerization (System Wide)
B7	AI Description Generator (UC-CAT-7)

EPIC C - Auction Service

ID	Task
C1	DB Schema Design for Auctions and Bids (UC1-AS, UC2-AS)
C2	Create Auction API (UC1-AS)
C3	Place Bid API (UC2-AS)

C4	Get Active Auctions API (UC4-AS)
C5	Bid Validation Logic (UC2-AS)
C6	Auction Closure Scheduler (UC3-AS)
C7	Determine Winner Logic (UC3-AS)
C8	Real-time Bid Notification (Pub-Sub Integration)

<u>EPIC D - Payment Service</u>

ID	Task
D1	DB Schema Design
D2	Payment API (UC-PAY-1)
D3	Order API (UC-PAY-1,UC-PAY-2)
D4	Shipping API (UC-PAY-1, UC-PAY-2)
D5	Receipt API (UC-PAY-2)
D6	Error Handling (UC-PAY-1)
D7	Receipt Sent Notification (Pub-Sub Integration)

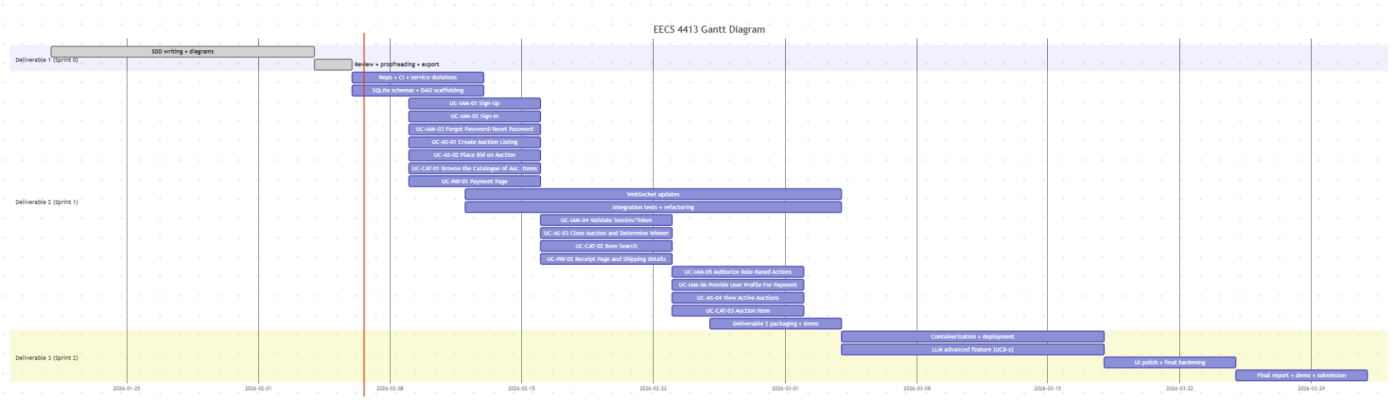
GANTT diagram:

(The diagram may be unclear due to its large size. Therefore, I have provided a Google Drive link to the PNG for better resolution.

https://drive.google.com/file/d/1aAVKnE-WW3yjkkAI6CozNTGCzwK8xUE7/view?usp=sharing)

<u>Legend</u>

Horizontal axis	Project timeline
Rows	Deliverable/sprint phases
Task bars	Scheduled tasks
Bar length	Task duration



Group meeting logs: Group met on January 30th to discuss who is completing what service, project logistics, and overall next steps/steps of completion. The group decided to create a WhatsApp group chat as the primary means of communication to coordinate logistics and update each other on project progress.

Anton will complete the catalogue service, Ravneet will complete the Identity and Authorization Management (IAM) Service, Syed will complete the Auction service, and Peyton will complete the Payment Service.

Meeting time: 20 minutes. Future meetings/communication for Deliverable 1 will be via WhatsApp call/texting.

Test Driven Development

(Ravneet Test Case 1)

Test ID	TC-GW-IAM-01
Category	IAM (via Gateway) – Sign-Up
Requirements Coverage	UC-IAM-01 Sign-Up (Success)
Initial Condition	Gateway and IAM are running; username/email do not already exist.
Procedure	1. Send POST /api/auth/signup with valid user + shipping address + role. 2. Observe HTTP status and response body.
Expected Outcome	HTTP **200** with JSON including message, userId, username, and role.
Notes	Confirms gateway forwarding works and IAM persists user.

(Ravneet Test Case 2)

Test ID	TC-GW-IAM-02
Category	IAM (via Gateway) – Sign-Up
Requirements Coverage	UC-IAM-01 Sign-Up (Duplicate Username/Email)
Initial Condition	A user already exists with the same username and email from TC-GW-IAM-01.

Procedure	1. Send the same POST /api/auth/signup payload again. 2. Observe status.
Expected Outcome	HTTP **409 Conflict** with JSON error message (e.g., “Username already exists”).
Notes	Verifies uniqueness enforcement and proper error propagation through gateway.

(Ravneet Test Case 3)

Test ID	TC-GW-IAM-03
Category	IAM (via Gateway) – Sign-In
Requirements Coverage	UC-IAM-02 Sign-In (Success)
Initial Condition	User exists (created in TC-GW-IAM-01).
Procedure	1. Send `POST /api/auth/login` with correct username/password. 2. Capture returned token.
Expected Outcome	HTTP 200 with JSON containing token, expiresIn, userId, username, role.
Notes	Token is used in later tests as Authorization: Bearer <token>.

(Ravneet Test Case 4)

Test ID	TC-GW-IAM-04

Category	IAM (via Gateway) – Sign-In
Requirements Coverage	UC-IAM-02 Sign-In (Failure – Wrong Password)
Initial Condition	User exists (created in TC-GW-IAM-01).
Procedure	1. Send POST /api/auth/login with correct username but wrong password. 2. Observe response.
Expected Outcome	HTTP 401 Unauthorized with JSON error message (e.g., “Invalid username or password”).
Notes	Confirms authentication rejection and error status forwarding.

(Ravneet Test Case 5)

Test ID	TC-GW-IAM-05
Category	IAM (via Gateway) – Token Validation
Requirements Coverage	UC-IAM-04 Validate Session/Token (Valid)
Initial Condition	Valid token obtained from TC-GW-IAM-03.
Procedure	1. Send POST /api/auth/validate with header Authorization: Bearer <token>. 2. Observe JSON result.
Expected Outcome	HTTP 200 with JSON: valid=true, and correct userId/username/role.

Notes	Confirms token is accepted by IAM and identity claims are returned.

(Ravneet Test Case 6)

Test ID	TC-GW-IAM-06
Category	IAM (via Gateway) – Token Validation
Requirements Coverage	UC-IAM-04 Validate Session/Token (Invalid Token)
Initial Condition	Gateway and IAM running.
Procedure	1. Send POST /api/auth/validate with header Authorization: Bearer not.a.real.token. 2. Observe JSON result.
Expected Outcome	HTTP 200 with JSON: valid=false, and null identity fields.
Notes	This is your current design choice (return 200 with valid=false). Some systems return 401; either is acceptable if consistent and documented.

(Ravneet Test Case 7)

Test ID	TC-GW-IAM-07
Category	IAM (via Gateway) – Role Authorization
Requirements Coverage	UC-IAM-05 Authorize Role-Based Actions (Authorized)

Initial Condition	Valid token for a BUYER user from TC-GW-IAM-03.
Procedure	1. Send GET /api/auth/authorize?requiredRole=BUYER with Authorization: Bearer <token>. 2. Observe response.
Expected Outcome	HTTP 200 with JSON: authorized=true and requiredRole=BUYER.
Notes	Confirms role checking works.

(Ravneet Test Case 8)

Test ID	TC-GW-IAM-08
Category	IAM (via Gateway) – Role Authorization
Requirements Coverage	UC-IAM-05 Authorize Role-Based Actions (Not Authorized)
Initial Condition	Valid token for BUYER user.
Procedure	1. Send GET /api/auth/authorize?requiredRole=SELLER with same BUYER token. 2. Observe response.
Expected Outcome	HTTP 200 with JSON: authorized=false and requiredRole=SELLER.
Notes	Confirms negative authorization case.

(Ravneet Test Case 9)

Test ID	TC-GW-IAM-09

Category	IAM (via Gateway) – Input Validation
Requirements Coverage	UC-IAM-01 Sign-Up (Validation Failure)
Initial Condition	Gateway and IAM running.
Procedure	1. Send POST /api/auth/signup with missing required field(s) (script uses missing password). 2. Observe response.
Expected Outcome	HTTP 400 Bad Request with validation JSON identifying the missing field (e.g., password must not be blank).
Notes	Confirms server-side validation + friendly error payload.

(Ravneet Test Case 10)

Test ID	TC-GW-IAM-10
Category	Gateway (IAM proxy) – Request Parameter Handling
Requirements Coverage	UC-IAM-05 Authorize Role-Based Actions (Bad Request)
Initial Condition	Gateway and IAM running; token exists (script attempts with token).
Procedure	1. Send GET /api/auth/authorize without requiredRole query param. 2. Observe response.
Expected Outcome	HTTP 400 Bad Request indicating missing required parameter.

Notes	If your response currently includes a large stack trace, you can clean it by setting server.error.include-stacktrace=never in gateway properties.

(Ravneet Test Case 11)

Test ID	TC-GW-IAM-11
Category	IAM via Gateway — User Profile Retrieval
Requirements Coverage	UC-IAM-06 Provide User Profile For Payment
Initial Condition	User has been created (sign-up succeeded) and a valid userId exists.
Procedure	1. GET /api/users/{userId} using the userId returned by signup/login. 2. Observe HTTP response and JSON body.
Expected Outcome	200 OK with JSON containing userId, username, email, firstName, lastName, role, and shippingAddress fields.
Notes	This is the endpoint Payment will use to populate shipping address on the payment page.

(Ravneet Test Case 12)

Test ID	TC-GW-IAM-12
Category	IAM via Gateway — Forgot Password

Requirements Coverage	UC-IAM-03 Forgot Password / Reset Password (Token Generation)
Initial Condition	User exists in the database (created in earlier steps).
Procedure	1. POST /api/auth/password/forgot with body { "username": "<username>" }. 2. Observe response.
Expected Outcome	200 OK with JSON containing "message": "Password reset token generated" and a resetToken.
Notes	Returning resetToken is suitable for demos/tests. In a real system this would be emailed, not returned in response.

(Ravneet Test Case 13)

Test ID	TC-GW-IAM-13
Category	IAM via Gateway — Reset Password
Requirements Coverage	UC-IAM-03 Forgot Password / Reset Password (Reset Success)
Initial Condition	A valid resetToken was generated from TC-GW-12 and is not expired/used.
Procedure	1. POST /api/auth/password/reset with body { "resetToken": "<token>", "newPassword": "<newPassword>" }. 2. Observe response.
Expected Outcome	200 OK with JSON { "message": "Password reset successful" }.

Notes	Confirms the reset token is accepted and password hash is updated.

(Ravneet Test Case 14)

Test ID	TC-GW-IAM-14
Category	IAM via Gateway — Sign-In (After Password Reset)
Requirements Coverage	UC-IAM-02 Sign-In (Success after password change)
Initial Condition	Password reset completed successfully (TC-GW-13).
Procedure	1. POST /api/auth/login with username and the new password. 2. Observe response.
Expected Outcome	200 OK with JSON containing a new token, expiresIn, userId, username, and role.
Notes	Confirms password reset actually took effect.

(Anton Test Case 1)

Test ID	TC-CAT-01
Category	Database Search
Requirements Coverage	UC 2.1 - Item Search
Initial Condition	System is running, Catalogue DB contains a record with status 'active'.

Procedure	1. Navigate to the Search bar. 2. Enter desired string 3. Click the Search button.
Expected Outcome	The system returns a result list containing the desired string with its initial price and end date.
Notes	Tests the keyword matching logic, and ensures retrieval of static catalogue data.

(Anton Test Case 2)

Test ID	TC-CAT-02
Category	Catalogue - Item Management
Requirements Coverage	UC 7 - Auction Item (Static Catalogue)
Initial Condition	User is logged in as a Seller, and the Catalogue DB is connected.
Procedure	1. Enter description. 2. Set Start Price 3. Set Duration: exactly 48 hours 4. Click 'Post Auction'.
Expected Outcome	A new entry is created in the DB with an End Date exactly 48 hours from the current time.
Notes	Verifies the auction setup, shipping cost storage, and End Date calculation.

(Anton Test Case 3)

Test ID	TC-CAT-03
Category	Catalogue - Business Logic
Requirements Coverage	UC 2 - Display Items (Filtering)
Initial Condition	The database contains an ‘expired item’ where the End_Date has passed.
Procedure	1. User performs a general search or browses the "All Items" category.
Expected Outcome	The expired item is not included in the returned list of items.
Notes	Ensures the Catalogue only serves items whose time hasn't expired.

(Anton Test Case 4)

Test ID	TC-CAT-04
Category	Catalogue - Input Validation
Requirements Coverage	UC 7 - Auction Item (Validation)
Initial Condition	User is on the 'Sell Item' page.
Procedure	1. Enter Description: "Test". 2. Enter Start Price: -10 (Or any negative value). 3. Click 'Submit'.

Expected Outcome	The system prevents submission and displays an error: "Initial price must be a positive integer."
Notes	Ensures that auction items must be sold at positive integers.

(Mustafa Test Case 1)

Test ID	TC-AUC-01
Category	Auction Service - Bid Placement
Requirements Coverage	UC2-AS-PlaceBid-Success
Initial Condition	User is authenticated, auction exists with status "Active", current highest bid is $50 (example).
Procedure	1. User selects an active auction 2. User enters bid amount of $75 3. User submits bid 4. User validates bid is higher than the current highest bid 5. User records bid and updates the auction
Expected Outcome	Bid is accepted, the current highest bid is updated to $75, user receives a confirmation message with the bid ID.
Notes	Bid amount must be strictly greater than the current highest bid. The previous highest bidder should be notified that they've been outbid.

(Mustafa Test Case 2)

Test ID	TC-AUC-02
Category	Auction Service - Bid Validation
Requirements Coverage	UC2-AS-PlaceBid-BidTooLow
Initial Condition	User is authenticated, auction exists with status "Active", current highest bid is $100 (example)
Procedure	1. User selects an active auction 2. User enters bid amount of $95 (lower than the current highest) 3. User submits bid 4. System validates bid amount
Expected Outcome	Bid is rejected, error message displayed: "Bid must be higher than current bid of $100", auction state remains unchanged.
Notes	System should validate bid amount before any database operations to prevent invalid data.

(Mustafa Test Case 3)

Test ID	TC-AUC-03
Category	Auction Service - Auction Creation
Requirements Coverage	UC1-AS-CreateAuction-Success
Initial Condition	User is authenticated with seller role, item exists in catalogue database.

Procedure	1. Seller navigates to "Create Auction" page 2. Seller enters: itemId="12345", startPrice=50, duration=24 hours, reservePrice=100 3. Seller submits form 4. System validates seller credentials and item existence 5. System creates auction record
Expected Outcome	Auction is created with status "Active", endTime = currentTime + 24 hours, auction ID is returned, seller receives confirmation message.
Notes	System must validate: seller has proper authorization, item exists in catalogue, startPrice and duration are positive values, reservePrice >= startPrice.

(Mustafa Test Case 4)

Test ID	TC-AUC-04
Category	Auction Service - Auction Closure
Requirements Coverage	UC3-AS-CloseAuction-WithWinner
Initial Condition	Auction exists with status "Active", endTime has passed, highest bid ($150) exceeds reserve price ($100)
Procedure	1. System scheduler detects auction end time has passed 2. System retrieves auction details and the highest bid 3. System compares the highest bid to the reserve price

	4. System determines the winner and closes the auction
Expected Outcome	Auction status changes to "Closed", highest bidder is marked as winner, the winner and the seller are notified, payment process is initiated with the Payment Service.
Notes	If highest bid is below reserve price, auction should close with the status "No Sale" and no winner declared. Seller should be notified

(Peyton Test Case 1)

Test ID	TC-PAY-01
Category	Payment Service - Complete Payment
Requirements Coverage	UC-PAY-1, UC-PAY-2
Initial Condition	Auction Is won by the user.
Procedure	1. Click Pay now 2. Enter Payment details 3. Click submit 4. Wait for payment processing
Expected Outcome	Payment is processed successfully, and the user is redirected to the receipt page.

(Peyton Test Case 2)

Test ID	TC-PAY-02

Category	Payment Service - Missing Field
Requirements Coverage	UC-PAY-1, UC-PAY-2
Initial Condition	Auction Is won by the user
Procedure	1. Click Pay now 2. Enter Payment details other than cvv 3. Click submit
Expected Outcome	Payment is not processed successfully and an error message is displayed indicating that all fields must be filled with valid information. The user remains on the payment page. No receipt is generated.

(Peyton Test Case 3)

Test ID	TC-PAY-03
Category	Payment Service - Invalid Card information
Requirements Coverage	UC-PAY-1,UC-PAY-2
Initial Condition	Auction Is won by the user
Procedure	1. Click Pay now 2. Enter invalid payment details 3. Click submit
Expected Outcome	Payment starts to process but an error message appears stating that the payment is declined check card details. The user stays on

	the payment page. No receipt is generated.

(Peyton Test Case 4)

Test ID	TC-PAY-04
Category	Payment Service - Receipt Generation and Display
Requirements Coverage	UC-PAY-1,UC-PAY-2
Initial Condition	Payment successfully processed. Shipping cost $5.99, and the item cost $15.00
Procedure	1. Payment successful 2. Receipt page generates
Expected Outcome	All valid information is displayed, including amount of days shipping will take and the total amount paid of $20.99, a sum of shipping and item cost.

Acknowledgements

AI was used in this deliverable to assist with brainstorming use case scenarios, giving feedback on UML diagrams according to course conventions, and proofreading technical documentation according to the success criteria. All design decisions, architectural choices, and implementations were made independently by the team members. AI-generated suggestions were critically evaluated and modified to meet project requirements.

AI Usage for IAM Service

- AI for this service was used primarily as a tool for offering suggestions when it came to industry best practices, aided with debugging when errors appeared during development, and helped with test development. During development, when the question of how the server will verify the client's identity appeared, AI was consulted on the different methods that one can use to achieve this. JWT design was one of the methods offered. AI was used to give an explanation and examples of how one implements JWT into a project. Using the examples provided, I developed an idea of how I wanted to structure the format of my token for the JWT that I was implementing into the IAM service. Once I created my token, I had AI evaluate it to see what improvements I can make to it. Through the examples AI had provided, I decided on how I would want to deliver the token to the client, and what protected routes will require a user to provide their token. AI was used when simulating suggestions for different ways to persist data as the service is running. AI aided in the process of developing a global exception handler class for the IAM service. It helped with generating an example on how to format an exception handler for my service. I used that example as a format on how to create the remaining exception handlers. AI helped with showcasing examples on how to generate a gateway service to function as a facade. It gave me an idea on how to create the gateway service and connect it to the other services. Due to limited

bash experience, AI was helpful in the development of test cases to check if the service's endpoints and features were functioning properly. In conclusion, AI was a major asset in the speed of iteration, showcasing industry best practices for web-development, and aiding in troubleshooting problems. A limitation that was seen by AI use was that suggestions can be version-sensitive, as import packages can vary between Spring versions. Therefore, one import that is not supported by the versions of the pre-existing packages can create errors.

AI Usage for Payment Service

- AI was used to first help get an idea what the structure of this service should look like. Once it looked good I then used ai to generate the project. I had to manually fix some of the code myself and I also consulted the AI to fix issues I saw pop up. What ended up happening was I got put into a loop and the AI couldn't fix the issue so I had to figure out the issue myself. AI was then used to help extend my service to communicate with the others and help generate code to do so. It also helped me with testing to make sure the code worked and pinpoint why it wasn't working. Overall all is very useful for generating a lot of code very fast but you end up spending a lot of time debugging and fixing errors so it's important to understand the code the ai generated.

AI Usage for Catalogue Service

- AI was primarily used to conduct file structure implementation, and how I could implement the different designs without overcomplicating the intended use of design patterns. Code was then manually written and refined to meet test requirements. AI also saved me lots of time when it came to searching different documents/other students' endpoints and variables, describing what components do what, thus ensuring swift and efficient progress in service production. It also saved me lots of time when it came to debugging, finding out what functions should do what, and how to actually implement the test cases pertaining to the use cases when all the other different services are also involved.

AI Usage for Auction Service

- AI was mainly used as a development aid during the implementation of the Auction service. It provided suggestions on how the service could be structured and what endpoints should exist for handling auction related actions such as creating auctions, placing bids, and retrieving auction data. AI was also useful when debugging issues that appeared during development, particularly with request handling and service logic. In addition, AI was used to generate example implementations for validating auction rules, such as ensuring that a new bid is higher than the current highest bid and that bids are only accepted while an auction is active. These examples were used as references and then adapted to fit the specific requirements of the project. AI also helped with generating and understanding test cases to verify that endpoints behaved correctly. Overall, AI helped speed up development and debugging, but the generated code still required manual review and adjustments to ensure it worked correctly within the project.

Tools used for the sequence, activity and component diagrams were as follows:

https://sequencediagram.org

https://lucid.app/lucidchart

https://www.umlet.com

https://app.diagrams.net/

https://plantuml.com/

And the document/report was compiled on Google Docs.